

Resolução de Problemas por meio de Buscas em Inteligência Artificial.



O Problema da Decisão

- Como um computador, que só segue regras, decide o que fazer?
- Analogia: Imagine que você está em um labirinto. Você não vê o mapa. Como você faz para sair?
- Você tenta um caminho. Se der errado, volta e tenta outro.
- **Isso é busca.**
- Conceito: Em IA, "resolver um problema" é encontrar uma sequência de ações que leve do ponto inicial (Estado Inicial) ao ponto desejado (Estado Objetivo).

Definição Formal do Problema

- Componentes Essenciais:
- Estado Inicial: Onde tudo começa. (Ex.: Casa do jogador em um jogo).
- Ações (Operadores): O que posso fazer? (Ex.: Andar para frente, virar à direita).
- Modelo de Transição: Se eu fizer Ação X no Estado Y, onde vou parar?
- Teste de Objetivo: Como sei que cheguei lá? (Ex.: Chegou na bandeira final).
- Custo de Caminho: Quanto gastei? (Ex.: Tempo, dinheiro, distância).

O Espaço de Estados

- Analogia: O "Mapa do Tesouro".
- O conjunto de todas as configurações possíveis que o problema pode ter.
- Exemplo: No jogo da velha, o espaço de estados são todas as grades de 3x3 possíveis (vazias, X e O).
- Objetivo: Navegar nesse espaço, indo do estado inicial para o final.

Árvore de Busca

- Analogia: Uma árvore genealógica de possibilidades.
- Raiz: Estado Inicial.
- Ramos: Ações possíveis.
- Nós (Folhas): Estados resultantes.
- Importante: A árvore pode ser gigantesca. A busca é a estratégia para não se perder nela.

Classificação das Buscas

- Divisão Principal:
- Busca Sem Informação (Cega): O agente não sabe qual caminho é melhor. Só sabe o que está à frente. É como procurar um endereço em uma cidade estranha sem GPS.
- Busca Com Informação (Heurística): O agente tem um "palpite" de qual direção é a melhor. É como ter um GPS que mostra o caminho mais curto, mas sem saber o trânsito.

Critérios de Avaliação de Busca

- Para escolher a melhor busca, avaliamos 4 coisas:
- Completude: Acha a solução se ela existir?
- Otimização: Acha a melhor solução (mais curta/barata)?
- Complexidade de Tempo: Quanto tempo leva?
- Complexidade de Espaço: Quanta memória consome?

Fronteira (Open List)

- Conceito Crucial: O algoritmo não "vê" tudo. Ele mantém uma lista de nós "na fila" para serem explorados.
- Analogia: Você está em um museu. Você não pode ver todas as salas de uma vez. Você tem uma lista de "próximas salas a visitar". Essa lista é a Fronteira.
- A diferença entre os algoritmos está na ordem em que eles organizam essa fila (FIFO, LIFO, ou por nota).

Resumo da Primeira Parte

- Problema = Estado Inicial + Ações + Objetivo.
- Resolver = Encontrar o caminho no Espaço de Estados.
- Tipos de Busca: Cega (sem dica) vs. Informada (com dica).

- **"Vamos agora aprender os dois principais métodos de busca sem informação: a preguiçosa e a teimosa."**

Busca Sem Informação (Cega)

- 
- Busca em Largura (BFS) - Breadth-First Search
 - Analogia da Onda: Jogue uma pedra em um lago. As ondas se expandem igualmente para todos os lados.
 - Como funciona: Explora todos os nós a uma profundidade "1", depois todos os nós a profundidade "2", depois "3"...

BFS

- É Uma árvore sendo varrida nível por nível.
- Estrutura de Dados: Fila (Queue) - FIFO (First In, First Out).
- Analogia da Fila: É como uma fila de banco. Quem chegou primeiro (nó pai) é atendido (expandido) primeiro, gerando seus filhos, que vão para o final da fila.

BFS - Vantagens e Desvantagens

- Vantagem: Garante o caminho mais curto (ótimo).
- Desvantagem: Consome MUITA memória.
- Analogia da Memória: Imagine explorar um palácio explorando todos os corredores do primeiro andar antes de subir para o segundo. Você precisa lembrar de todos os corredores do primeiro andar ao mesmo tempo.

Busca em Profundidade (DFS) - Depth-First Search

- Analogia do Labirinto: Você vai por um corredor até o fim (até bater numa parede). Se não deu certo, você volta um pouco (backtrack) e tenta a próxima bifurcação.
- Como funciona: Vai fundo em um caminho antes de explorar os irmãos.



DFS - Visualização

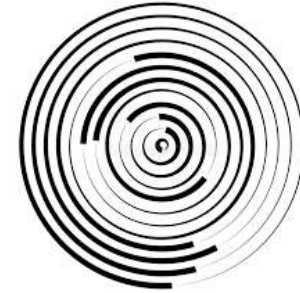
- Uma árvore sendo varrida descendo pela esquerda até a folha, depois sobe, depois desce na próxima.
- Estrutura de Dados: Pilha (Stack) - LIFO (Last In, First Out).
- Analogia da Pilha: É como uma pilha de pratos. O último prato que você colocou em cima é o primeiro que você vai pegar (o filho mais recente é explorado primeiro).

DFS - Vantagens e Desvantagens

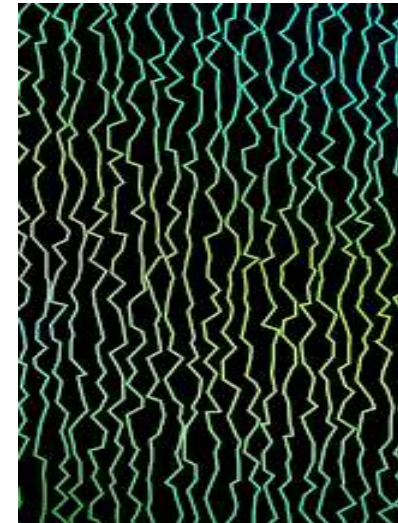
- Vantagem: Consome pouca memória.
- Desvantagem: Não garante o caminho mais curto. Pode ficar "perdido" em um caminho infinito.
- Analogia: É a pessoa teimosa que entra em um túnel escuro e só volta quando vê que não tem saída, mesmo que houvesse uma saída fácil do lado de fora da entrada.

Comparação Visual: BFS vs DFS

- BFS: Círculos concêntricos saindo do centro.

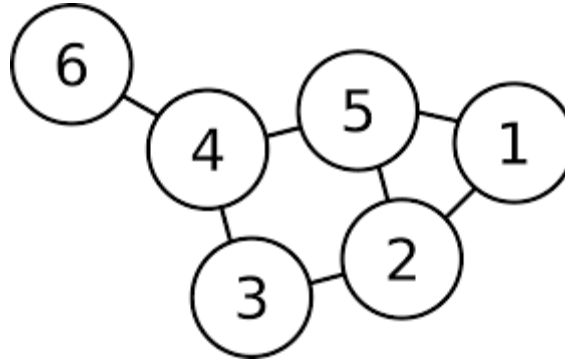


- DFS: Linha torta indo direto para o fundo.



Problema: Ciclos e Estados Repetidos

- Problema: E se o grafo tiver um loop? (Ex.: Sala 1 -> Sala 2 -> Sala 1).



- Risco: O algoritmo pode ficar preso para sempre (loop infinito).
- Solução: Manter uma lista de "Estados Visitados" (Closed List). Antes de entrar em um estado, verifico se já estive lá.

Busca de Custo Uniforme (UCS)

- Variação importante: E se as ações têm custos diferentes (andar 1km custa 1, andar 10km custa 10)?
- Funcionamento: Expande o nó com o menor custo acumulado.
- Analogia: É o BFS, mas em vez de olhar para o "nível" (profundidade), olha para o "preço da conta" até agora.

Resumo Busca Cega

- BFS: Garante o menor número de passos. Usa Fila. Memória alta.
- DFS: Vai fundo. Usa Pilha. Memória baixa, mas não é ótimo.
- UCS: Garante o menor custo. Usa Fila de Prioridade (custo).

Exemplo Prático - Quebra-Cabeça (8-puzzle)

- Cenário: Embaralhar números em um tabuleiro 3x3.

- | | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 0 |
- **Movimentos Permitidos:** O espaço vazio (0) pode trocar de lugar com uma peça adjacente (cima, baixo, esquerda, direita).
- ← Espaço vazio

- Questão: Usar BFS nesse problema? Impossível! O espaço de estados é enorme (milhões de estados). BFS ficaria sem memória (explosão combinatória).
- Conclusão: Precisamos de uma "dica" para guiar a busca. Entramos na Busca Informada.

- "Até agora andamos cegos. Vamos agora colocar um 'GPS' no nosso algoritmo."

Busca com Informação (Heurística)

- O que é Heurística? (do grego "heuriskein" = encontrar, descobrir):
- Definição: É um "palpite educado". Uma função que estima o custo do nó atual até o objetivo.
- Notação: $h(n)$:
- $h(n)$ = custo estimado de n até o objetivo
- $h(\text{objetivo}) = 0$ (chegou, não falta nada)
- Analogia: É a "distância em linha reta" no Waze.
- O Waze não sabe se a rua está bloqueada (obstáculo), mas ele sabe que, em linha reta, São Paulo está a 400km. Isso ajuda a descartar caminhos que vão para o lado oposto

Exemplo de Heurística

- Problema: Cidade de mapas.
- $h(A)$ = Distância em linha reta de A até o destino.
- Problema: 8-puzzle (quebra-cabeça).
- h_1 = Número de peças fora do lugar.
- h_2 = Soma das distâncias de São paulo (distância que cada peça precisa andar).

Busca Gulosa (Best-First)

- Estratégia: Expande o nó que parece mais próximo do objetivo (menor $h(n)$).
- Analogia: É a pessoa que olha para a montanha e só anda na direção dela, ignorando rios, pântanos ou estradas.
- Problema: Pode ser enganada por obstáculos locais. Não é ótima.

Busca A (A Estrela)*

- O algoritmo mais famoso de busca.
- Fórmula: $f(n) = g(n) + h(n)$
- $g(n)$: Custo real do início até o nó n (o que já gastei).
- $h(n)$: Heurística (estimativa do que falta).
- Analogia: Você está em um carro.
- g : Quilômetros que você já rodou.
- h : Distância em linha reta que falta.
- f : Melhor palpite de viagem total.

Por que o A é Especial?*

- Condição: Se a heurística for admissível (nunca superestima o custo real) e consistente, o A^* é ótimo e completo.
- Analogia: O A^* equilibra a preguiça (BFS) e a teimosia (DFS/Gulosa). Ele não quer só o caminho mais curto (g), nem só o que parece mais perto (h). Ele quer o melhor equilíbrio entre "onde estou" e "para onde vou".

Exemplo Visual - Cidade

- Mapa: Nós (cidades) e arestas (distâncias).
- BFS: Vai expandindo em círculos. Demora.
- Gulosa: Vai direto para a cidade que parece mais perto do destino. Pode cair em um beco sem saída.
- A*: Vai pelas cidades que minimizam (caminho percorrido + estimativa restante). Acha a rota ideal.

Os Três Critérios de Avaliação

- Antes de comparar os algoritmos, precisamos entender o que cada critério significa:

Critério	Pergunta que Responde	Por Que é Importante
Compleitude	O algoritmo sempre encontra uma solução se ela existir?	Não adianta ser rápido se nunca achar a resposta!
Otimização	O algoritmo encontra a MELHOR solução (mais curta/mais barata)?	Muitas vezes queremos a melhor rota, não qualquer rota
Memória	Quanto espaço de armazenamento o algoritmo consome?	Computadores têm memória limitada

Comparação de Desempenho

Algoritmo	Compleitude	Otimização	Memória
BFS	Sim	Sim (passos)	Alta
DFS	Não (ciclos)	Não	Baixa
Gulosa	Não	Não	Média
A*	Sim	Sim (custo)	Alta (mas controlada)

Analogia Final - Escolhendo a Ferramenta

- Labirinto simples, saída próxima: DFS (rápido, memória baixa).
- Grafo pequeno, quero o caminho mais curto: BFS.
- Caminhos com custos diferentes (ex.: trânsito): UCS ou A*.
- Espaço de estados enorme (ex.: jogos): A* com uma boa heurística.

Desafios Reais

- Explosão Combinatória: O número de possibilidades cresce exponencialmente.
- A explosão combinatória é o fenômeno onde o número de possibilidades cresce **exponencialmente** (ou fatorialmente) à medida que o tamanho do problema aumenta.
- Imagine um fungo que dobra de tamanho a cada dia:
 - Dia 1: 1 cm²
 - Dia 2: 2 cm²
 - Dia 3: 4 cm²
 - Dia 10: 512 cm²
 - Dia 20: 524.288 cm² (52 metros quadrados!)
 - Dia 30: 536.870.912 cm² (53.687 metros quadrados - mais de 5 campos de futebol!)
 - O crescimento exponencial começa devagar, mas rapidamente se torna incontrolável.
- Nesses casos, a busca exaustiva é impossível. Usamos versões otimizadas (A* com limites de memória - IDA*) ou heurísticas muito fortes.

- "Vimos como explorar. Mas quem decide qual ação tomar baseado no que encontramos?"

Implementação e Contexto em IA

- Estrutura de Dados na Prática
- Node (Nó): Objeto que guarda:
- Estado
- Nó pai (quem me gerou)
- Ação (como cheguei aqui)
- Custo g (até aqui)
- Profundidade

Pseudocódigo Genérico de Busca

- Inicialize a fronteira com o estado inicial.
- Inicialize a lista de visitados vazia.
- Enquanto fronteira não estiver vazia:
 - a. Remova um nó da fronteira (ordem depende do algoritmo).
 - b. Se for o objetivo, retorne sucesso.
 - c. Adicione o nó aos visitados.
 - d. Expanda o nó (gere filhos).
 - e. Para cada filho: se não está em visitados nem na fronteira, adicione à fronteira.

O Papel da Fronteira

- Fronteira = Estratégia.
- Se a fronteira é uma Fila (FIFO) = Busca em Largura.
- Se a fronteira é uma Pilha (LIFO) = Busca em Profundidade.
- Se a fronteira é uma Fila de Prioridade (ordem por $f(n)$) = A^* .

Aplicações no Mundo Real

- GPS (Waze/Google Maps): A^* com heurística de distância em linha reta + trânsito em tempo real (custo g variável).
- Jogos (IA de inimigos): A^* para encontrar o jogador em mapas (NavMesh).
- Roteamento de Redes: Como enviar um pacote de dados do Brasil para o Japão da forma mais rápida.
- Robótica: Planejamento de movimento (como mover o braço robótico sem colidir).

Exemplo Clássico - Problema do Missionário e Canibais

- Problema: 3 missionários e 3 canibais devem atravessar um rio num barco que cabe 2. Se canibais superarem missionários em qualquer margem, eles os comem.
- Solução: É um espaço de estados. Usamos BFS ou A* (heurística: número de pessoas na margem inicial) para achar a sequência de travessias.

Resumo - Mapa Mental

- Resolução de Problemas
 - Busca Cega
 - BFS (Fila, Ótimo, Memória)
 - DFS (Pilha, Rápido, Não Ótimo)
 - Busca Informada
 - Gulosa (h)
 - A* ($g + h$) -> O algoritmo

Dicas

- Não decore fórmulas, entenda a fila: Se você entender se o algoritmo usa Fila ou Pilha, você entendeu 80% da diferença entre BFS e DFS.
- A Heurística é o "chute": Quanto melhor o chute, mais rápido o A^* .
- Pratique no papel: Desenhe árvores para problemas pequenos (ex.: 3 cidades, 8-puzzle simplificado). A visualização manual é essencial.

- Pergunta 1: Qual algoritmo você usaria para encontrar a rota mais curta em um mapa onde todas as ruas têm o mesmo tamanho?
- Pergunta 2: Se você tem um palpite muito bom sobre a distância até o destino, qual algoritmo combina o custo real com esse palpite?
- Pergunta 3: Se a memória do computador é muito limitada, mas o tempo não é problema, qual algoritmo prefere?
- Frase Final: "A IA não adivinha o futuro; ela explora possibilidades de forma inteligente. A busca é a base para o pensamento computacional estratégico."

Exercícios

- Identifique os 5 componentes formais de um problema de busca no cenário abaixo:
- "Você está em uma estação de metrô (Estação A) e quer chegar na Estação D. Você pode pegar trens que vão para as estações adjacentes. Cada trecho entre estações leva 5 minutos."

